Tagless Final

eason: 7 · Advanced FP · Category: Functional · Level: Advanced · Build with Waq

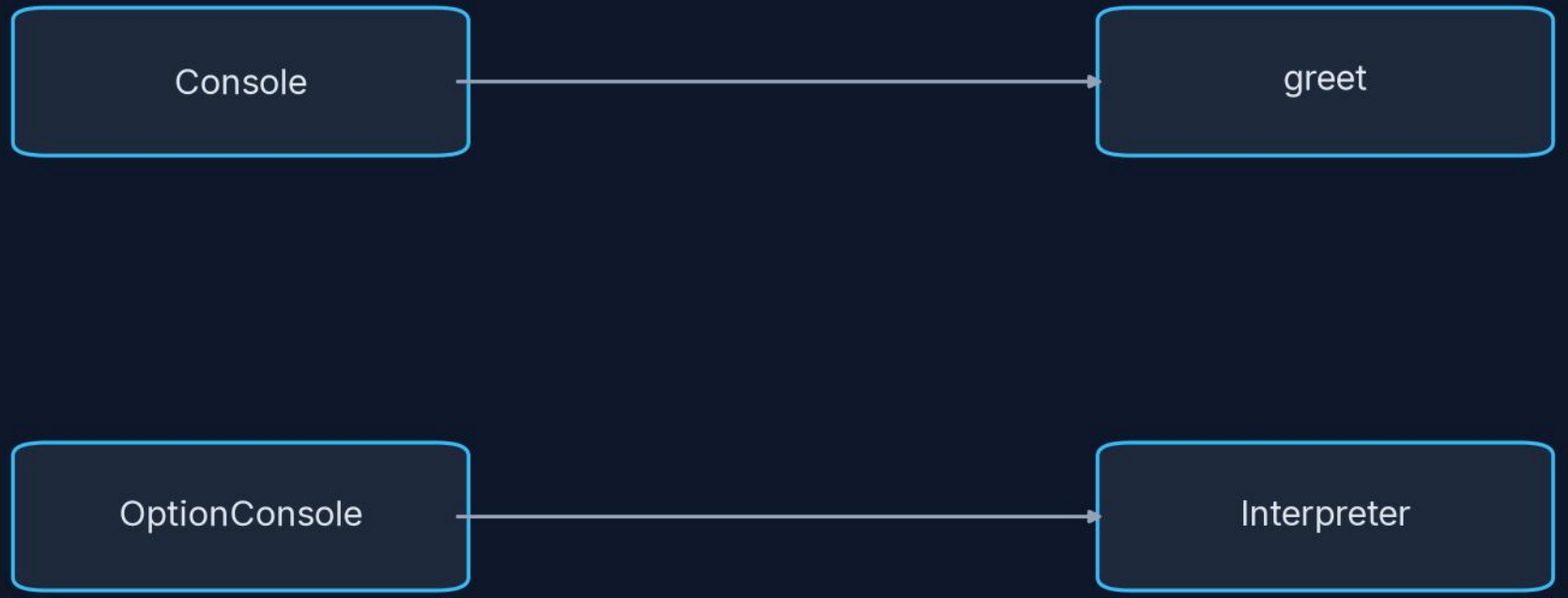
Today's Hook

- Today: Tagless Final as a StreamForge product force — not trivia.
- Runnable Demo on JDK 25 with captured stdout.
- Classic encoding → Java 25 / Zazr modernization → trade-offs.
- CapCut slides + full spoken script included.

The Problem

- Naive local fixes: more ifs, statics, nulls, or threads.
- Illegal states still compile; tests become order-dependent.
- Design reviews lack a shared name for the force.
- Interviews punish memorization without judgment.

Structure



Tagless Final structure

Real-World Scenario

- StreamForge creator platform is the running metaphor.
- Same names appear in naive code, refactor, and Demo output.
- Stay in one story for the full 10–12 minutes.
- Product language first; pattern vocabulary second.

Naive Code

```
// Naive sketch for Tagless Final
public class Naive {
  // duplicated logic / leaked details / unsafe sharing
  Object doIt(Object... args) {
    if (args == null) return null;
    return args[0];
  }
}
```

Why Naive Hurts

- Coupling spreads across controllers and infrastructure.
- Tests need sleeps or static reset hooks.
- New variants require shotgun edits.
- On-call cannot reason about failure modes.

Implementation

```java
// file: src/main/java/com/buildwithwaqar/patterns/functional/tagless/TaglessFinal.java
package com.buildwithwaqar.patterns.functional.tagless;

import com.buildwithwaqar.patterns.functional.typeclass.Kind;
import com.buildwithwaqar.patterns.functional.typeclass.Monad;
import com.buildwithwaqar.patterns.functional.typeclass.OptionKind;
import dev.zazr.control.Option;

/**
 * Tagless Final: programs are written against an algebra (interface of capabilities),
 * not a concrete effect type. Interpreters provide the algebra for Option, IO, etc.
 */
public final class TaglessFinal {
    private TaglessFinal() {}

    /** Unit type — Zazr Option rejects null, so Void is awkward; use a singleton. */
    public enum Unit { INSTANCE }

    public interface Console<F> {
        Kind<F, String> readLine();
        Kind<F, Unit> printLine(String s);
    }
```

Implementation (2)

```
    public static <F> Kind<F, String> greet(Monad<F> M, Console<F> console) {
        return M.flatMap(console.printLine("What is your name?"), ignored ->
                M.flatMap(console.readLine(), name ->
                        M.flatMap(console.printLine("Hello, " + name), ignored2 ->
                                M.pure(name))));
    }

    /** Test interpreter using Option as a trivial effect (pre-canned input). */
    public static final class OptionConsole implements Console<OptionKind.Witness> {
        private final String input;
        public OptionConsole(String input) { this.input = input; }

        public Kind<OptionKind.Witness, String> readLine() {
            return OptionKind.lift(Option.some(input));
        }

        public Kind<OptionKind.Witness, Unit> printLine(String s) {
            return OptionKind.lift(Option.some(Unit.INSTANCE));
        }
    }
}
```

Demo.main

```
package com.buildwithwaqar.patterns.functional.tagless;
import com.buildwithwaqar.patterns.functional.typeclass.OptionKind;
public final class Demo {
  public static void main(String[] a) {
    System.out.println("=== Tagless Final walkthrough ===");
    var console = new TaglessFinal.OptionConsole("Waqar");
    var result = TaglessFinal.greet(OptionKind.MONAD, console);
    System.out.println("name=" + OptionKind.lower(result).get());
  }
}
```

Walkthrough Output

```
=== Tagless Final walkthrough ===
name=Waqar
```

Step-by-Step Trace

- Each stdout line maps to a refactor stage.
- Re-run Demo locally on JDK 25 to verify.
- Differences usually mean a dirty tree — pull main.
- Pause the video and predict the next line before revealing.

Modern Java 25 / Zazr

```
// Java 25 / Zazr direction for Tagless Final
// records + sealed + switch
// Option/Either/Validation instead of null
// virtual threads for blocking fan-out
// inject interfaces at the edge
```

Trade-offs

- Clarity vs ceremony — do not spawn five types for two lines.
- Closed sealed worlds vs open SPI plugins.
- Teaching demos omit production idempotency/metrics.
- Prefer simpler language features when they erase the force.

Related Patterns

- Contrast with the neighbor pattern (force difference).
- Name the roles from memory in sixty seconds.
- Point to the season map for what comes next.
- Link the GitHub episode folder in the description.

Recap

- Force → naive pain → Tagless Final → Demo stdout.
- Modernize with Java 25 / Zazr when it shrinks code.
- Teach when NOT to use it.
- Practice: break a test, fix it, re-run Demo.